Part 1

웹 개발 가이드

단일 컴퓨터에서 웹 서비스까지

1장. 단일 컴퓨터
2장. 네트워크
3장. 인터넷 서비스
4장. 서버 가동과 운영
5장. 서비스 추상화
6장. 웹 서비스 구조
7장. 현대적 분업 구조

1장. 단일 컴퓨터

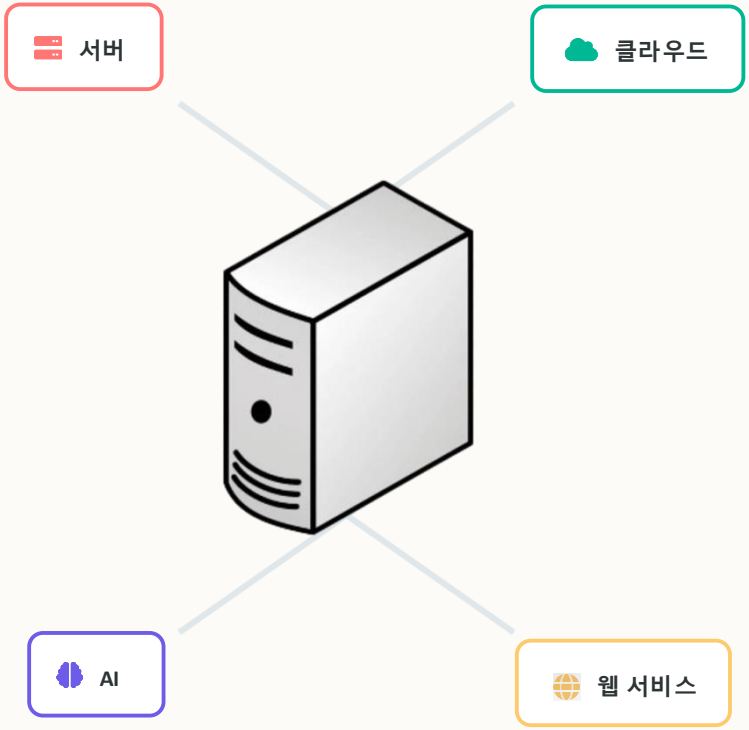
모든 화려한 서비스의 실체는 컴퓨터 하나입니다.

1장

모든 서비스의 출발점: 컴퓨터 한 대

모든 IT 서비스는 서버, 클라우드, AI라는 멋진 이름을
가지고 있지만,
물리적으로는 **"컴퓨터"**라는 실체 위에서 동작합니다.

- **서버** = 고성능 컴퓨터
- **클라우드** = 여러 대의 컴퓨터
- **AI** = 컴퓨터에서 실행되는 프로그램
- **웹 서비스** = 컴퓨터 간의 통신



1장

컴퓨터 구조 – 폰 노이만 구조

CPU (중앙처리장치)

계산하는 두뇌
실제 연산을 수행

RAM (메모리)

임시 작업대
빠르게 데이터 접근

SSD/HDD (저장장치)

영구 보관 창고
데이터 장기 보존

도서관 비유: 작동 흐름

서가 (SSD)
책 꺼내기

→

책상 (RAM)
펼쳐놓기

→

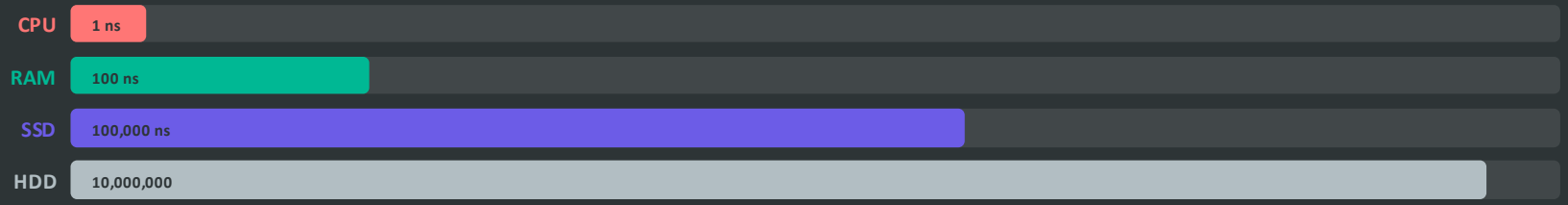
사람 (CPU)
읽고 이해하기

1장

CPU / RAM / SSD(HDD)의 역할 비교

	CPU	RAM	SSD/HDD
역할	연산 처리	임시 저장	영구 저장
속도	매우 빠름	빠름	느림
휘발성	-	전원 OFF시 삭제	데이터 유지

속도(레이턴시) 비교: 숫자가 작을수록 빠릅니다



핵심: RAM은 CPU보다 100배 느리고, SSD는 RAM보다 1,000배 느립니다!

1장

저장 → 메모리 → 연산 → 저장의 순환

SSD/HDD
데이터 꺼내기
(Load)

RAM
메모리 적재
(Ready)

CPU
연산 처리
(Process)

RAM
결과 임시 저장
(Hold)

SSD/HDD
최종 저장
(Save)

웹 요청 처리

- 요청 데이터를 메모리에 올림
- CPU가 로직 처리
- 결과를 DB(저장소)에 저장

서버 응답

- DB에서 데이터 읽기
- 메모리에서 가공 (JSON 변환)
- 사용자에게 전송

AI 추론

- 모델을 메모리에 로드
- GPU/CPU가 계산 수행
- 결과값 반환 및 저장

1장

프로그램과 프로세스의 차이

프로그램

Program (Static)

- 디스크에 저장된 **파일**
- 아직 실행되지 않은 상태
- 죽어있는 코드 덩어리

비유

요리 레시피 (책)

실행 (Execute)

프로세스

Process (Dynamic)

- 메모리에 올라간 **실체**
- CPU와 메모리를 사용 중
- 살아서 움직이는 상태

비유

요리 중인 냄비

왜 중요할까요?

서버 장애, 메모리 누수, 트래픽 폭증은 모두 정적인 파일이 아니라 **"실행 중인 프로세스 관리 문제"**에서 발생합니다.

1장

서버의 본질: 고성능 컴퓨터

서버는 특별한 기계가 아닙니다.
본질적으로 **일반 컴퓨터와 동일한 구조**입니다.

구분	일반 컴퓨터	서버
구조	CPU + RAM + SSD	CPU + RAM + SSD
운영 시간	필요할 때만 켬	**24시간 365일 가동**
성능	일반 사양	**고성능 사양**
네트워크	가끔 연결	**항상 연결**
신뢰성	일반	**고신뢰성 설계**

✔ **서버는 '고성능·고신뢰 컴퓨터'일 뿐입니다.**

2장. 네트워크

컴퓨터들이 서로 대화하기 시작할 때 서비스가 탄생합니다.

2장

컴퓨터 간의 통신이란?

네트워크는 단순히 "연결"이 아니라
다음 **네 가지 핵심 질문**에 대한 답을 찾는 과정입니다.

누가?

송신자

편지 비유

보내는 사람
주소

1

어디

수신자

편지 비유

받는 사람
주소

2

무엇을?

데이터

편지 비유

편지 내용
(정보)

3

어떻게?

프로토콜

편지 비유

우편 시스템
규칙

4

2장

IP DNS 포트

DNS (도메인)

"강남아파트" (이름)

예: www.google.com

IP 주소

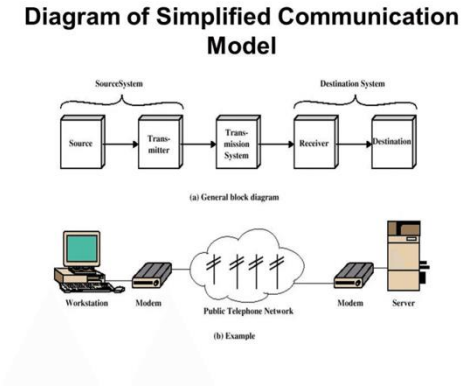
"서울시 강남구 테헤란로 123" (실제 주소)

예: 142.250.196.78

포트 (Port)

"101동 505호" (구체적 위치)

예: :80 (웹), :443 (보안)



핵심 정리

IP = 컴퓨터의 위치 (어디로)

포트 = 프로그램 구분 (누구에게)

DNS = 전화번호부 (이름 → 주소)

2장

프로토콜이란? – 통신의 약속

프로토콜(Protocol) = 통신 규칙

서로 다른 시스템이 소통하기 위해 미리 정해둔 약속입니다.

규칙이 없을 때

"안녕하세요" vs "Hello"
서로 해석 불가 → **통신 실패**

규칙(프로토콜)이 있을 때

"Hello" ↔ "Hello"
공통 언어 사용 → **통신 성공**

주요 프로토콜 예시

HTTP

Web Page Transfer

웹 페이지를
주고받는 규칙

TCP

Reliable Transfer

데이터를 빠짐없이
안전하게 전송

FTP

File Transfer

파일을
전송하는 규칙

SMTP

Email Transfer

이메일을
보내는 규칙

2장

JSON이란? – 데이터 표현의 공통 언어

JSON (JavaScript Object Notation)

사람이 읽기 쉽고, 컴퓨터가 처리하기 쉬운
데이터 표현 규칙입니다.

- 키(Key)와 값(Value)의 쌍
- 중괄호 {} 로 데이터를 감쌈
- 대괄호 [] 로 목록(배열) 표현
- 웹과 서버 통신의 표준 언어

```
{
  "이름":"김철수",
  "나이":30,
  "직업":"개발자",
  "취미":[
    "독서",
    "운동",
    "코딩"
  ]
}
```

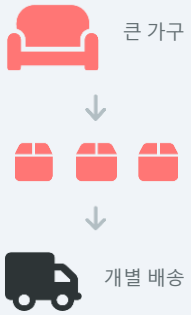
← 키 (Key)
← 값 (Value)
← 배열 (Array)

2장

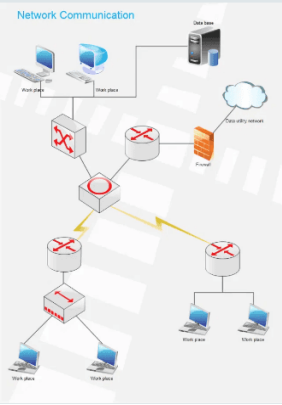
컴퓨터 간 통신 원리 – 패킷 전송

데이터는 한 덩어리로 보내지지 않고 '패킷(Packet)'이라는 작은 단위로 쪼개져 전송됩니다.

가구 배송 비유



패킷 전송 원리



복잡한 네트워크망을 통해
패킷들이 각자 최적의 경로로 이동합니다.

1. **분할 (Split)** — 데이터 쪼개기
2. **전송 (Send)** — 독립적 이동
3. **조립 (Assemble)** — 순서대로 맞춤
4. **확인 (Check)** — 누락 재요청

3장. 인터넷 서비스

단순 연결이 '가치'로 변하는 순간

3장

연결이 서비스가 되는 순간

단순한 네트워크 연결은 “통신”일 뿐입니다.
하지만 여기에 가치가 결합되면 “서비스”가 됩니다.

단순 연결 (통신) + 기능과 가치 = 인터넷 서비스

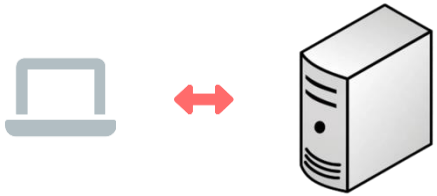
지속성	다수 사용자	요청 처리 로직	데이터 관리

인터넷 서비스란 “연결 위에 기능과 가치를 얹은 구조”입니다.

3장

1:1 통신의 한계

1:1 통신은 두 컴퓨터가 동시에 연결되어 있어야 하며, 상대가 없으면 아무 것도 할 수 없습니다.



동시 접속 불가

확장 불가

관리 복잡성

사용자 수가 늘어날수록 연결 수가 기하급수적으로 증가하고, 확장성과 안정성이 완전히 무너집니다.

3장

클라이언트-서버 구조

"모든 요청은 서버가 받고, 서버가 처리해서 응답한다"는
중앙 집중형 모델입니다.

동시 접속 처리 | 데이터 관리 | 보안 통제

오늘날 인터넷 서비스의 거의 모든 구조는 이 모델을 기반으로 합니다.

3장

배달 서비스로 비유한 클라이언트-서버

주문 (요청)

→

←

음식 (응답)

손님

클라이언트

가게

서버

사용자는 음식을 직접 만들지 않고 주문만 합니다. 가게가 요리를 담당하고 사용자는 결과만 받습니다.
이처럼 요청과 처리의 분리가 클라이언트-서버 구조의 핵심입니다.

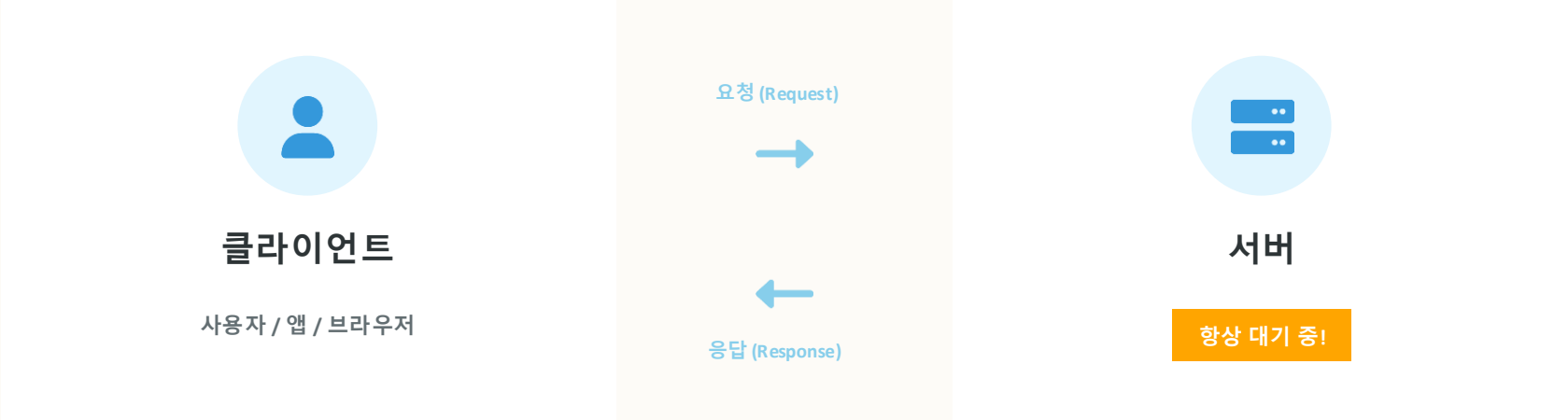
3장

배달 서비스로 비유한 클라이언트-서버

사용자는 음식을 직접 만들지 않고 주문만 합니다. 가게가 요리를 담당하고 사용자는 결과만 받습니다.
이처럼 요청과 처리의 분리가 클라이언트-서버 구조의 핵심입니다.

3장

요청/응답 모델 원리



인터넷 서비스는 본질적으로 "요청 → 처리 → 응답"의 반복 구조입니다.
이 모델은 웹, 모바일 앱, API, AI 서비스까지 전부 동일합니다.

3장

REST 개념 소개

REST는 웹 서비스를 설계하는 가장 직관적인 약속입니다.

자원 (Resource)

모든 것을 URL로 표현함
(예: /users/1)

행위 (Verb)

HTTP 메서드로 조작함
(GET, POST 등)

표현 (Representation)

데이터를 주고받는 형태
(JSON, XML 등)

URL(자원) + HTTP Method(행위) = 직관적인 API

'URL은 자원, HTTP 메서드는 행위'라는 관점으로 설계하면 서비스가 확장 가능해집니다.

3장

웹의 등장과 진화

과거: 문서 공유

연구 문서를 공유하기 위한 시스템이었습니다.
단순한 정보 전달의 도구로 시작되었습니다.

현재: 서비스 플랫폼

쇼핑, 금융, 게임, AI까지!
모든 서비스를 실현하는 강력한 인프라가 되었습니다.

→ 서버 / 인프라

웹은 단순한 기술이 아니라 서비스를 실현하는 인프라입니다.

4장. 서버 가동과 운영

서비스로 바뀌는 코드

4장

서비스의 작동 원리

코드를 작성한다 → 서버에 올린다 → 실행된다 → 요청을 처리한다

서비스는 코드만 작성한다고 되는 것이 아니라
실행 환경 + 운영 구조가 반드시 필요합니다.

서버 가동은 서비스 구현의 **절반 이상**을 차지합니다.

4장

서버에 파일을 올려 실행한다는 것

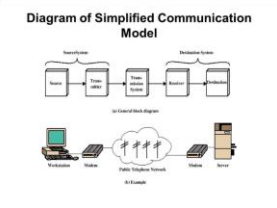
로컬 개발 환경

개발자의 PC에서
테스트하는 환경

VS

서버 실행 환경

실제 사용자가 접속하는
서비스 환경



[참고: 데이터가 전달되는 통신 모델 구조]

개발자는 로컬 PC에서 테스트하지만,
실제 서비스는 “멀리 있는 서버”에서 실행됩니다.

이 둘의 환경 차이가 장애와 버그의 주요 원인이 됩니다.

4장

SSH와 FTP

SSH

제어 (Control)

서버에 접속해
명령어를 실행하는 도구

FTP

전송 (Transfer)

서버와 내 컴퓨터 사이의
파일을 옮기는 도구

운영자는 항상 "멀리 있는 서버"를 다룬다는 개념이 중요합니다.

4장 서버 OS – Linux

Linux는 왜 전 세계 서버의 표준 OS가 되었을까요?

무료 (Open)

비용 부담 없이
어디서나 사용 가능

안정성

수개월 동안 꺼지지 않고
작동하는 견고함

자원 효율성

불필요한 기능 없이
핵심 성능에 집중

자동화 친화성

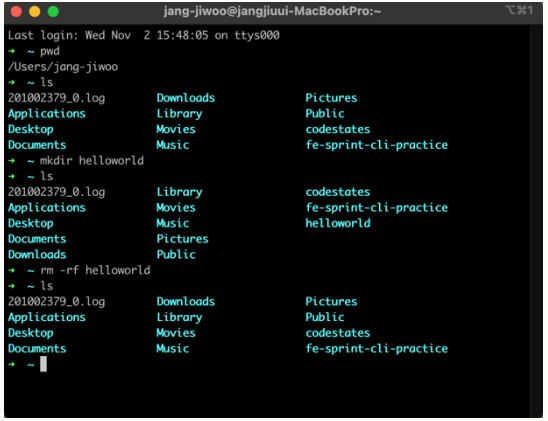
명령어 기반으로
대규모 운영에 최적

$ sudo systemctl start service → GUI보다 CLI 중심의 최적화된 운영

가용성, 자원 효율성, 안정성 덕분에 Linux는 서버 운영의 표준이 되었습니다.

4장

CLI와 GUI



CLI

텍스트 명령어를 직접 입력해 빠르고 자동화된 제어

장점
- 빠르다.
- 자원(메모리) 소모가 작다.
- 자동화에 적합하다. (scripting)
- 세부적이고 다양한 기능을 활용할 수 있다.

단점
- 사용 초기 진입 장벽이 크다.

GUI

아이콘, 마우스 클릭, 창 등을 활용하여 직관적이고 사용하기 쉬운 시각적 환경

장점
- 초보 유저가 사용하기 쉽다.
- 명령어가 없어도 직관적으로 사용 가능하다.

단점
- 느리다.
- 자원(메모리 등) 소모가 크다.
- 자동화가 불편하다.

4장

포그라운드 vs 백그라운드 실행

포그라운드

- ✓ 터미널에서 직접 실행
- ✗ 터미널 종료 시 함께 종료
- 개발 및 테스트 용도

비유

실시간 전화 통화

터미널 종료 시

→

←

백그라운드

- ✓ 터미널과 독립적으로 실행
- ✓ 터미널 종료해도 계속 실행
- 실제 서비스 운영 용도

비유

자동 응답기

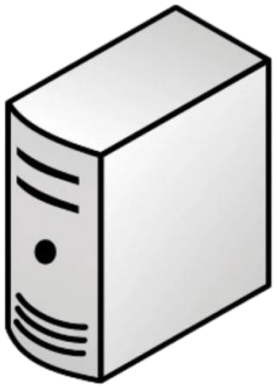
서버는 24시간 깨어있어야 합니다!

관리자가 접속을 끊어도 서비스가 멈추지 않도록, 실제 운영 환경에서는 항상 **백그라운드 실행**이 필수입니다.

4장

실제 운영 관점: 로그와 모니터링

서버 운영의 핵심은
"문제 없이 오래 유지하는 것"입니다.



1. 로그 (Log)

서버에서 발생한 모든 이벤트를 기록합니다.
문제가 생겼을 때 원인을 찾는 '블랙박스' 역할을 합니다.

2. 모니터링 (Monitoring)

CPU, 메모리, 트래픽 상태를 실시간으로 감시합니다.
서버의 건강 상태를 체크하는 '종합 검진'입니다.

3. 알림 (Alert)

문제가 감지되면 즉시 담당자에게 알립니다.
장애를 조기에 발견하여 피해를 최소화합니다.

문제가 발생하기 전에 미리 감지하고 대응하는 것이 운영의 핵심입니다.

4장

장애 대응과 복구: 완벽한 시스템은 없다

1. 장애 발생
예기치 못한 시스템 오류 발생

2. 탐지 (Detection)
모니터링 시스템의 자동 알림

3. 원인 분석 (Analysis)
로그 확인 및 문제 지점 파악

4. 긴급 조치 (Action)
서비스 재시작 등 즉각적 정상화

5. 근본 해결 & 방지
코드 수정 및 재발 방지 대책 수립

복구 시간 목표 (RTO)

5분 이내	우수
30분 이내	양호
1시간 이상	개선 필요

운영에서 중요한 것은 장애를 '막는 것'보다
'얼마나 빨리 복구하느냐'입니다.
장애는 반드시 발생한다는 전제하에
시스템을 설계해야 합니다.

5장. 서비스 추상화

복잡함을 숨기고 유연한 확장을 얻는 기술

5장

왜 나눠야 하는가? 계층 구조의 필요성

모놀리식 (Monolithic)

비유: 한 방에 모든 물건 쌓기

- ✕ 처음엔 쉽지만 갈수록 복잡해짐
- ✕ 한 곳의 문제가 전체로 번짐
- ✕ 유지보수가 매우 어려움

계층 구조 (Layered)

비유: 서랍장에 분류해서 정리

- ✓ 확장과 유지보수가 용이함
- ✓ 장애가 다른 곳으로 퍼지지 않음
- ✓ 팀 단위로 나누어 개발 가능

모든 것을 하나로 만들면 처음엔 쉽지만, 서비스가 커지면 **관리의 재앙**이 됩니다.
지속 가능한 서비스를 위해 **역할에 따른 계층 분리**는 필수적인 전략입니다.

5장

추상화란 무엇인가: 복잡함을 감추는 기술

추상화(Abstraction)란 복잡한 내부 구조는 감추고,
사용자에게 꼭 필요한 인터페이스만 드러내는 것입니다.

자동차

운전자는 엔진의 폭발 행정을 몰라도 핸들과 페달만으로 운전할 수 있습니다.

스마트폰

사용자는 내부 반도체 회로를 몰라도 터치 스크린만으로 앱을 사용합니다.

ATM

고객은 은행의 복잡한 전산망을 몰라도 버튼 몇 개로 입출금을 합니다.

IT에서의 추상화 사례

- API: 내부 코드를 감춤
- DB: 저장 방식을 감춤
- 클라우드: 서버 관리를 감춤

추상화는 복잡도를 낮추어 인간이 거대한 시스템을 다룰 수 있게 해주는 마법입니다.

5장

계층 분리의 4가지 이유: 서비스 생존 전략

계층 분리는 단순한 구조 미화가 아니라 **비즈니스의 안정성을 지키는 핵심 전략** 입니다.

1. 장애 격리

한 계층의 문제가 다른 계층으로 번지는 것을 막아 서비스 전체가 멈추는 대참사를 방지합니다.

예: 데이터베이스 장애 시에도 사용자 화면은 정상 작동

2. 보안 강화

계층별로 접근 권한을 철저히 통제하여 핵심 데이터에 대한 직접적인 외부 노출을 차단합니다.

예: 일반 사용자는 DB 서버에 직접 접근 불가

3. 유지보수 용이

특정 계층만 수정하거나 교체해도 다른 계층에 미치는 영향이 적어 개발 속도가 빨라집니다.

예: 디자인(UI) 변경 시 서버 로직 수정 불필요

4. 유연한 확장성

전체 시스템을 건드리지 않고도 트래픽이 몰리는 특정 계층만 독립적으로 확장할 수 있습니다.

예: 데이터 처리량이 늘어날 때 DB 성능만 업그레이드

5장

결합도와 응집도: 좋은 시스템의 조건

낮은 결합도

Low Coupling

비유: 레고 블록

- 각 블록은 독립적으로 존재함
- 쉽게 조립하고 분리할 수 있음
- 한 블록이 망가져도 교체 가능

높은 응집도

High Cohesion

비유: 기계 부품

- 각 부품은 명확한 한 가지 역할 수행
- 관련된 기능끼리 긴밀하게 모여 있음
- 내부적으로는 매우 튼튼하게 연결됨

좋은 시스템은 "서로 덜 의존(낮은 결합도)"하면서,
"자기 역할은 확실히(높은 응집도)" 하는 구조입니다.

5장

Scale Up vs Scale Out: 서버 확장 전략

확장은 단순히 서버를 늘리는 것이 아니라 서비스의 생존과 비용을 결정하는 구조적 선택 입니다.

Scale Up

수직 확장

비유

버스를 더 큰 버스로 교체

- ✓ 설정이 매우 간단함
- ✗ 물리적 성능 한계가 있음
- ✗ 업그레이드 시 서비스 중단 가능

Scale Out

수평 확장

비유

버스를 여러 대 운행

- ✓ 무한에 가까운 확장 가능
- ✓ 장애 시 다른 서버가 대체
- ✗ 구조가 복잡해짐

Scale In

축소

비유

한산할 때 버스 대수 줄이기

- ✓ 불필요한 비용 절감
- ✓ 트래픽에 따른 유연한 대응
- ℹ 클라우드의 핵심 장점

현대 클라우드 전략의 핵심은 **Scale Out** 입니다. 서버 한 대의 성능보다 **여러 대로 분산하는 것** 이 훨씬 효율적입니다.

5장

클라우드 vs 온프레미스: 소유에서 임대로

온프레미스 (On-Premise)

"내 집 마련"

서버를 직접 구매하고 데이터센터를 운영하며 모든 것을 직접 소유하고 관리합니다.

클라우드 (Cloud)

"호텔/공유 오피스"

필요한 만큼 서버 자원을 빌려 쓰고 사용한 만큼만 비용을 지불합니다.

구분	온프레미스	클라우드
소유	직접 소유	**임대 (빌려 쓰기)**
비용	초기 투자 비용 큼	**사용한 만큼 지불**
유연성	확장 시 수주~수개월 소요	**즉시 확장 가능**
관리	하드웨어까지 직접 관리	**대부분 자동화/추상화**

클라우드는 서버 관리를 추상화 하여, 사용자가 인프라 걱정 없이 서비스에만 집중 하게 합니다.

5장

모놀리식과 MSA: 덩어리 vs 조각들

모놀리식 (Monolithic)

비유: 하나의 큰 빌딩

- 모든 기능이 하나의 앱에 통합
- 구조가 단순하여 개발이 빠름
- 작은 규모의 서비스에 적합

마이크로서비스 (MSA)

비유: 여러 개의 작은 건물들

- 기능별로 독립된 서비스로 분리
- 구조가 복잡하지만 유연함
- 대규모 서비스 확장에 유리

구분	모놀리식	MSA
배포	전체 시스템 재배포 필요	서비스별 독립 배포 가능
장애 영향	한 곳의 장애가 전체로 확산	해당 서비스만 영향 (장애 격리)
확장성	시스템 전체를 확장해야 함	필요한 부분만 선택적 확장
복잡도	낮음 (관리가 쉬움)	높음 (운영 난이도 상승)

6장. 웹 서비스 구조

현대 인터넷 세상의 보편적 설계도

6장

웹 서비스란? 기능 중심의 소프트웨어

웹 서비스는 인터넷을 통해 제공되는 **기능 단위의 소프트웨어** 입니다.

구분	단순 웹페이지	웹 서비스
목적	정보 전달	기능 제공
상호작용	읽기만 가능 (Static)	입력/처리/저장 (Dynamic)
예시	뉴스 사이트, 블로그	쇼핑몰, 은행, SNS

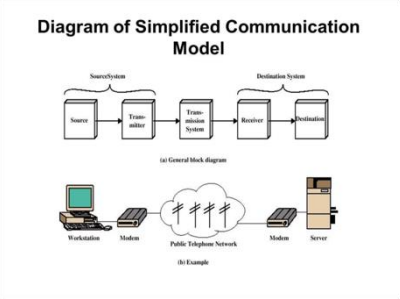
UI (화면)
사용자가 보는 인터페이스

로직 (규칙)
요청을 처리하는 비즈니스 규칙

데이터 (정보)
저장하고 관리하는 핵심 정보

통신 (전달)
데이터를 주고받는 통로

웹 서비스의 통신 모델



6장

웹 서비스의 기본 뼈대: 3계층 구조

거의 모든 웹 서비스는 역할에 따라 3가지 계층으로 나뉩니다.

01

프레젠테이션 계층 (Presentation)

사용자가 직접 보고 상호작용하는 **화면**입니다. (브라우저, 모바일 앱)

02

비즈니스 로직 계층 (Business Logic)

사용자의 요청을 처리하고 규칙을 적용하는 **두뇌**입니다. (서버 프로그램)

03

데이터 계층 (Data)

서비스에 필요한 모든 정보를 안전하게 보관하는 **창고**입니다. (데이터베이스)

이 구조는 서비스의 규모와 상관없이 **유지보수와 확장** 을 위한 가장 표준적인 설계 방식입니다.

6장

클라이언트-서버 모델: 요청과 응답의 대화

클라이언트 (Client)

비유: 손님

원하는 서비스를 요청하는 쪽

요청 (Request)

→

←

응답 (Response)

서버 (Server)

비유: 점원

요청을 처리하고 결과를 제공하는 쪽

웹 서비스의 모든 동작은 **"손님의 주문(요청)"** 과 **"점원의 서빙(응답)"** 이라는 끊임없는 대화로 이루어집니다.

6장

API란? 서비스 간의 약속된 창구

API(Application Programming Interface)는
프로그램들이 서로 대화하기 위한 약속입니다.

레스토랑 비유

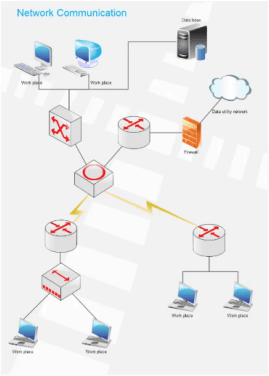
손님 (클라이언트): 주문을 하는 사용자

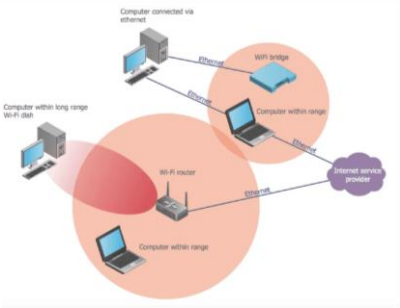
메뉴판 (API): 주문 가능한 목록과 규칙

주방 (서버): 요청을 처리하여 요리 제공

손님은 주방에서 어떻게 요리하는지 몰라도, 메뉴판(API)에 적힌 대로 주문하면 원하는 음식을 받을 수 있습니다.

실제 네트워크 통신 구조





API는 내부의 복잡한 구현을 감추고(추상화),
정해진 규칙대로만 요청하면 결과를 보장해주는 창구입니다.

6장

HTTP: 웹의 통신 규칙

요청 (Request)

```
GET /board/1 HTTP/1.1
Host: example.com
User-Agent: Chrome...
```

"1번 게시글 좀 보여줘!"

응답 (Response)

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "title": "안녕!", "content": "반가워" }
```

"여기 있어, 성공이야!"

200 OK: 성공

404 Not Found: 없음

500 Error: 서버 오류

HTTP 덕분에 전 세계의 서로 다른 시스템들이 **동일한 규칙** 으로 통신할 수 있습니다.

6장

CRUD: 웹 서비스 기능의 4대 원칙

웹 서비스가 다루는 거의 모든 기능은 CRUD라는 4가지 기본 동작의 조합으로 이루어집니다.

기능	의미	HTTP 메서드	실제 예시
Create	생성	POST	회원가입, 게시글 작성, 댓글 달기
Read	조회	GET	글 목록 보기, 내 정보 조회, 검색하기
Update	수정	PUT / PATCH	프로필 사진 변경, 게시글 내용 수정
Delete	삭제	DELETE	회원 탈퇴, 게시글 삭제, 댓글 지우기

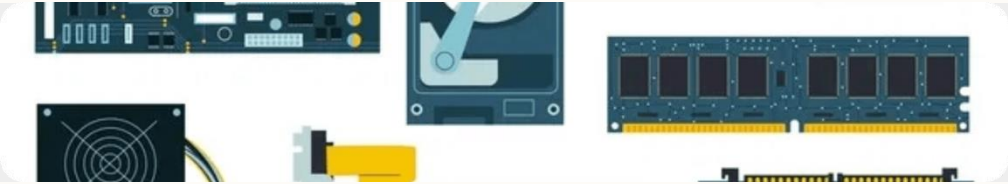
복잡해 보이는 모든 웹 서비스는 결국 이 4가지 동작을 어떻게 조합하느냐의 문제입니다.

6장

게시판 예시로 보는 웹 구조

게시판 기능과 기술 매핑

기능	CRUD	HTTP	URL 예시
글 작성	Create	POST	/api/articles
글 목록	Read	GET	/api/articles
글 보기	Read	GET	/api/articles/{articleId}
글 수정	Update	PUT/PATCH	/api/articles/{articleId}
글 삭제	Delete	DELETE	/api/articles/{articleId}



글 작성 처리 흐름

1. 사용자가 "글 작성" 버튼 클릭
2. 프론트엔드가 POST 요청 생성
3. 서버가 요청 수신 및 검증
4. DB에 새 게시글 저장
5. 서버가 성공 응답 반환
6. 프론트엔드가 완료 메시지 표시

이 모든 과정이 1초도 안 되는 시간에 일어납니다!

6장

요청 → 처리 → 응답 흐름과 에러 추적

서비스 처리 흐름

1. 사용자가 버튼 클릭
2. 프론트엔드가 HTTP 요청 생성
3. 서버가 요청 수신
4. 비즈니스 로직 처리 (권한/유효성)
5. 데이터베이스 접근 (조회/저장)
6. 결과 데이터 생성 (JSON)
7. HTTP 응답 반환
8. 프론트엔드가 화면에 표시

에러 추적 가이드

1-2단계 오류: 프론트엔드 문제

버튼이 안 눌리거나, 요청 자체가 안 나가는 경우

3-4단계 오류: 백엔드 로직 문제

서버가 요청을 받았으나 처리에 실패한 경우 (500 에러)

5단계 오류: 데이터베이스 문제

데이터를 저장하거나 불러오지 못하는 경우

8단계 오류: 렌더링 문제

데이터는 왔으나 화면에 제대로 안 그려지는 경우

흐름을 이해하면 에러가 발생했을 때 **어느 계층의 담당자를 찾아야 할지** 명확해집니다.

7장. 현대적 분업 구조

UI / FE / BE / DB / AI

7장 음식점 비유로 보는 전체 구조 총정리

웹 서비스	음식점 비유	주요 역할
UI	메뉴판	사용자가 선택 가능한 옵션을 시각적으로 표시
프론트엔드	홀 직원	손님의 주문을 받고, 완성된 음식을 전달하는 창구
백엔드	주방 (요리사)	주문을 분석하여 실제 요리(데이터 처리)를 수행
데이터베이스	재료 창고	신선한 재료(데이터)를 보관하고 필요할 때 제공
AI	디저트 전문점	추천 메뉴나 특수 기능 등 고도의 전문 서비스 제공

메뉴 선택 (UI) -> 주문 접수 (프론트엔드) -> 주방 전달 (백엔드) -> 재료 준비 (DB) -> 요리 중 (백엔드 처리) -> 서빙 (프론트엔드) -> 식사 (사용자)

웹 서비스는 각자의 역할이 명확한 전문가들의 협업 시스템입니다.
이 구조를 이해하면 문제가 생겼을 때 어디를 고쳐야 할지 명확히 알 수 있습니다.

7장 UI와 프론트엔드: 사용자 경험의 중심

UI (User Interface)

사용자가 보는 **화면 그 자체**를 설계합니다.
버튼의 위치, 색상, 폰트, 레이아웃 등 **디자인 중심**의 영역입니다.

프론트엔드 (Frontend)

UI를 **실제로 동작하게** 만드는 기술입니다.
사용자 입력을 처리하고 서버와 통신하여 **화면을 업데이트**합니다.

HTML (뼈대)
웹페이지의 구조와 내용을 정의합니다.

CSS (인테리어)
색상, 폰트, 애니메이션 등 디자인을 입힙니다.

JavaScript (동작)
클릭 반응, 데이터 처리 등 역동적인 기능을 만듭니다.

프론트엔드의 주요 책임 범위

- 버튼 클릭 이벤트 처리
- 입력 데이터 형식 검증
- 로딩 및 에러 화면 표시
- 서버 응답 데이터 화면 출력

프론트엔드는 사용자가 서비스를 만나는 첫 번째 관문이자 사용자 경험(UX)의 핵심입니다.

7장 백엔드: 서비스의 핵심 두뇌

백엔드(Backend)는 서비스의 **규칙과 판단이 모이는 핵심 두뇌** 입니다.

- 요청 처리 및 분석
- 권한 및 보안 검증
- 비즈니스 로직 적용
- 데이터 가공 및 변환
- 트랜잭션 관리
- 외부 시스템 연동

예: 회원가입 시 비밀번호 암호화, 결제 금액 검증, 게시글 작성 권한 확인 등 모든 중요한 판단을 수행합니다.

고성능 서버 인프라

백엔드는 프론트엔드가 믿을 수 없는 영역입니다. **모든 중요한 판단과 처리는 반드시 백엔드에서 수행** 되어야 서비스의 신뢰성이 유지됩니다.

7장 데이터베이스: 기억과 기록의 중심

데이터베이스(DB)는 단순한 저장 공간이 아니라 지능적인 데이터 관리 시스템 입니다.

중복 방지

동일한 데이터가 여러 번 저장되는 낭비를 막고 효율적으로 관리합니다.

무결성 유지

데이터가 서로 모순되지 않도록 일관성을 엄격하게 보장합니다.

빠른 조회

인덱스(색인) 기능을 통해 수억 개의 데이터 중에서도 즉시 찾아냅니다.

동시성 제어

수많은 사용자가 동시에 접근해도 데이터가 꼬이지 않게 관리합니다.

트랜잭션

작업 단위를 묶어 '전부 성공'하거나 '전부 실패'하게 하여 안전을 지킵니다.

보안 및 백업

허가된 사용자만 접근하게 하고, 만약의 사태에 대비해 복사본을 만듭니다.

관계형 DB (RDBMS)

MySQL, PostgreSQL 등 - 엑셀처럼 정해진 표 형식으로 저장

NoSQL

MongoDB, Redis 등 - 자유롭고 유연한 구조로 대량 데이터 처리

데이터가 손상되면 서비스 전체가 무너집니다. DB는 서비스의 신뢰성을 결정하는 심장 입니다.

7장

AI: 자동화와 지능화 계층

AI는 기존의 규칙 기반 로직이 처리하기 어려운 복잡한 판단을 담당하는 전문가입니다.

추천

사용자의 취향을 분석하여 상품이나 콘텐츠를 제안합니다.

분류

이미지 인식, 스팸 메일 필터링 등 데이터를 구분합니다.

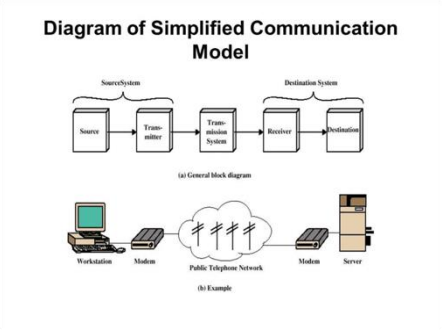
예측

과거 데이터를 바탕으로 수요나 이상 거래를 탐지합니다.

자동화

챗봇, 자동 응답 등 사람의 판단을 대신 수행합니다.

지능형 통신 및 판단 모델

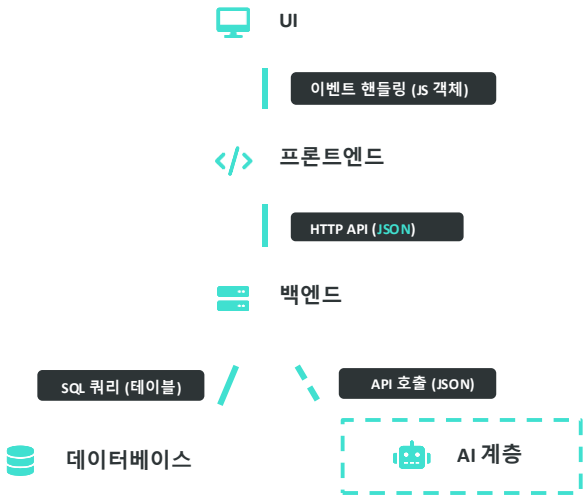


AI는 데이터의 흐름 속에서 핵심적인 판단을 내립니다.

실제 서비스 적용 예시

- 넷플릭스: 영화 추천
- 구글: 검색 결과 순위
- 은행: 이상 거래 탐지
- 쇼핑몰: 개인화 마케팅

7장 계층 간 소통 구조와 데이터 형식



통신 예시 (프론트 → 백)

```
POST /api/posts
{
  "title": "안녕하세요",
  "content": "반갑습니다"
}
```

통신 예시 (백 → DB)

```
INSERT INTO posts
(title, content)
VALUES
('안녕하세요', '반갑습니다')
```

이 구조 덕분에 각 계층은 독립적으로 발전하면서도 약속된 데이터 형식을 통해 하나의 서비스로 완벽하게 작동합니다.

7장

Docker: 환경의 통일과 컨테이너 배포

⚠"내 컴퓨터에서는 되는데 서버에서는 안 돼요!"

운영체제, 라이브러리 버전, 환경 변수 차이로 인해 발생하는 개발자의 고질적인 문제입니다.

일반 배포

짐을 하나씩 풀어서 옮기는 방식.
도착지의 환경이 다르면 짐이 망가지거나 작동하지 않을 수 있습니다.

Docker (컨테이너)

짐을 **컨테이너에 담아 통째로** 옮기는 방식.
어디로 옮기든 컨테이너 내부 환경은 동일하게 유지됩니다.

- 환경 통일
- 배포 자동화
- 무한 확장성

Docker는 "어디서나 동일하게 작동하는 환경" 을 보장하는 현대 인프라의 핵심 기술입니다.

SUMMARY

IT 서비스의 4대 핵심 기둥

컴퓨터

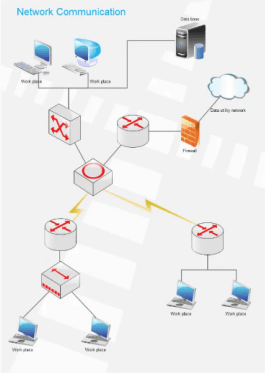
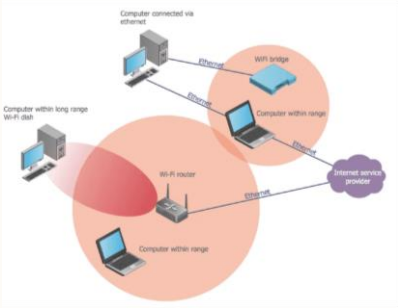
CPU, RAM, SSD의 조화로운 연산과 저장

네트워크

IP, 포트, 프로토콜을 통한 정확한 소통

서버

요청과 응답으로 이루어진 서비스의 본질

계층 분리

프론트, 백, DB, AI의 전문적인 협업